Q1 **Master Theorem: Four Subproblems** 4 marks

Analyze $T(n) = 4T(n/2) + n^2 \log n$ using the Master Theorem. Identify a, b, and f(n), state whether a standard case applies, and justify the asymptotic bound.

Your Answer

Given the recurrence relation: $T(n) = 4T(n/2) + n^2 \log n$

Identifying the parameters:
Comparing with the standard Master Theorem form $T(n) = aT(n/b) + f(n)$:

* a = 4 (number of subproblems)

Save Answer Next →

Q2 **Master Theorem: Unequal Growth** 4 marks

Analyze T(n) = 3T(n/2) + n using the Master Theorem, where applicable. Explain which term dominates.

Your Answer

The recurrence T(n) = 3T(n/2) + n can be analyzed using the Master Theorem, which applies to recurrences of the form T(n) = aT(n/b) + f(n).

Here, a = 3, b = 2, and f(n) = n.

Step 1: Compute n^(log_b a)

Save Answer Next →

Q3 **Master Theorem: Square-Root Work** 4 marks

Find the asymptotic complexity of T(n) = 2T(n/4) + √n. State the parameters and justify the applicable case.

Your Answer

Since a = 2, b = 4, and f(n)= Θ(n^(1/2)), the critical exponent log_4 2 = 0.5 matches f(n), which falls under Case 2 of the Master Theorem, resulting in a complexity of Θ(√(n)log n).

Save Answer Next →

Q4 **Master Theorem: Five Subproblems** 4 marks

A divide-and-conquer algorithm makes 5 recursive calls on subproblems of size n/3 and performs $\Theta(n^2)$ extra work. Determine its asymptotic time complexity.

Your Answer

The recurrence relation is $T(n) = 5T(n/3) + \Theta(n^2)$. Here, $a = 5$, $b = 3$, and $f(n) = \Theta(n^2)$. Comparing $f(n)$ to $n^{(\log_b a)}$, we find that $\log_3 5 \approx 1.465$. Since n^2 grows faster than $n^{1.465}$ (i.e., $f(n) = \Omega(n^{(\log_3 5 + \varepsilon)})$ for $\varepsilon \approx 0.535$), and the regularity condition $5(n/3)^2 \leq c \cdot n^2$ holds for $c = 5/9 < 1$, this falls under Case 3 of the Master Theorem. Therefore, the asymptotic time complexity is $\Theta(n^2)$.

Save Answer Next →

Q5 **Master Theorem: Logarithmic Factor** 4 marks

Analyze $T(n) = 16T(n/4) + n^2/\log n$. Discuss whether the standard Master Theorem applies directly and provide a justified bound.

Your Answer

To analyze the recurrence T(n)= 16T(n/4)+ (n^2)/(log n):

1. Identify the variables:
    * a = 16
    * b = 4
    * f(n)= (n^2)/(log n)
2. Calculate the critical exponent:

Save Answer Next →

Q6 **Substitution: Linearithmic Recurrence** 4 marks

Use the substitution method to prove an asymptotic bound for T(n) = 2T(n/2) + n. State an induction hypothesis and base case.

Your Answer

To determine the asymptotic bound for the recurrence T(n)= 5T(n/3)+ Θ(n^2) using the Master Theorem, we follow these steps:

1. Identify the parameters:
    * a = 5
    * b = 3
    * f(n)= Θ(n^2)
2. Calculate the critical exponent:

Save Answer Next →

Q7 **Substitution: Decreasing by One** 4 marks

Use substitution or expansion to establish a tight asymptotic bound for $T(n) = T(n-1) + n$, assuming $T(1)=1$.

Your Answer

To find the tight asymptotic bound for $T(n)= T(n - 1)+ n$ with $T(1)= 1$, we can use the method of expansion (iteration):

1. Expansion Step:

$T(n)= T(n - 1)+ n$

$T(n)=[T(n - 2)+(n - 1)]+ n$

$T(n)=[T(n - 3)+(n - 2)+(n - 1)]+ n$

...

Save Answer Next →

Q8 **Substitution: Halving with Constant Work** 4 marks

Prove that T(n) = T(n/2) + 1 is Θ(log n), assuming n is a power of two.

Your Answer

The recurrence relation is T(n)= 5T(n/3)+ Θ(n^2).

1. Identify variables: a = 5, b = 3, and f(n)= Θ(n^2).
2. Calculate critical exponent: log_b a = log_3 5 ≈ 1.465.
3. Compare f(n) with n^(log_b a): Since f(n)= n^2 and 2 > 1.465, f(n) grows polynomially faster than n^(log_3 5). Specifically, f(n)= Ω(n^(log_3 5+ϵ)) for ϵ ≈ 0.535.
4. Check Regularity Condition: For Case 3, we must satisfy a · f(n/b)≤ c · f(n) for some c < 1.

Save Answer Next →

Q9 **Substitution: Exponential Recurrence** 4 marks

Establish a tight asymptotic bound for T(n) = 2T(n−1) + 1 with T(0)=1. Explain the growth pattern.

Your Answer

To find the tight asymptotic bound for T(n)= 2T(n − 1)+ 1 with T(0)= 1, we can use the method of substitution (unrolling the recurrence):

1. Unrolling the recurrence:

T(n)= 2T(n − 1)+ 1

T(n)= 2 [2T(n − 2)+ 1]+ 1 = 2^2T(n − 2)+ 2 + 1

T(n)= 2^2 [2T(n − 3)+ 1]+ 2 + 1 = 2^3T(n − 3)+ 2^2 + 2^1 + 2^0

Save Answer Next →

Q10 **Check a Complexity Claim** 4 marks

An analyst claims T(n) = T(n/2) + n is Θ(n log n). Assess the claim and establish the correct asymptotic bound.

Your Answer

The claim that T(n)= T(n/2)+ n is Θ(n log n) is incorrect.
To assess this, we use the Master Theorem where:

* a = 1 (number of subproblems)
* b = 2 (division factor)
* f(n)= n (extra work)

1. Calculate the critical exponent: log_b a = log_2 1 = 0.

Save Answer Next →

Q11 **Algorithm Crossover** 4 marks

Algorithm A performs $5n \log_2 n$ operations and Algorithm B performs $0.02n^2$ operations. Compare them and estimate the crossover input size.

Your Answer

To find the crossover point, we set the two complexities equal to each other:
5n log_2 n = 0.02n^2
Dividing both sides by n (assuming n > 0):
5 log_2 n = 0.02n
Multiplying both sides by 50 to isolate the n term:
250 log_2 n = n

Save Answer Next →

Q12 **Scalability: n log n** 4 marks

A program processes 10,000 records in 2 seconds using an $O(n \log_2 n)$ algorithm. Estimate runtime for 100,000 records, assuming proportional scaling and negligible fixed overhead.

Your Answer

To estimate the runtime for 100,000 records, we use the proportional relationship for O(n log_2 n) complexity:
(T_2)/(T_1) = (n_2log_2 (n_2))/(n_1log_2 (n_1))
Given:
n_1 = 10,000, T_1 = 2 seconds
n_2 = 100,000
Calculation:

Save Answer Next →

Q13 **Scalability: Quadratic** 4 marks

An $O(n^2)$ algorithm takes 4 seconds for 2,000 items. Estimate runtime for 6,000 items, assuming the quadratic term dominates.

Your Answer

Since the algorithm is O(n^2), the runtime increases by the square of the factor by which the input size increases. The input size increases from 2,000 to 6,000, which is a factor of 3. Therefore, the runtime increases by a factor of 3^2 = 9. Given the original runtime is 4 seconds, the estimated runtime is 4 × 9 = 36 seconds.

Save Answer Next →

Q14 **Compare Three Growth Rates** 4 marks

Compare O(n), O(n log n), and $O(n^2)$ growth. Discuss how their operation counts scale and give an appropriate use case for each.

Your Answer

The recurrence relation is T(n)= 5T(n/3)+ Θ(n^2).
By applying the Master Theorem:

1. Identify parameters: a = 5, b = 3, and f(n)= Θ(n^2).
2. Calculate critical exponent: log_b a = log_3 5 ≈ 1.465.
3. Compare f(n) with n^(log_b a): Since n^2 grows faster than n^1.465, we have f(n)= Ω(n^(log_3 5+ϵ)) for ϵ ≈ 0.535.
4. Check Regularity Condition: For Case 3, we must verify if a · f(n/b)≤ c · f(n) for some c < 1.

Save Answer Next →

Q15 **Constant Factors and Big-O** 4 marks

Two algorithms take $T_1(n)=100n$ and $T_2(n)=n \log_2 n$. Explain why asymptotic notation alone may not determine which is faster for small inputs.

Your Answer

Asymptotic notation (Big-O) describes the behavior of functions as n approaches infinity, focusing on the growth rate rather than exact values. In this case, T_1(n)= 100n is O(n) and T_2(n)= n log_2 n is O(n log n). Asymptotically, T_1(n) is faster because linear growth is slower than linearithmic growth.
However, for small inputs, the constant factor becomes significant. For T_1(n), the constant is 100, whereas for T_2(n), the multiplier is effectively log_2 n. If n is small (for example, n = 2), T_1(2)= 200 while T_2(2)= 2 log_2 2 = 2. In this scenario, T_2(n) is much faster. Asymptotic notation ignores these constant coefficients and lower-order terms, which can cause the theoretically "slower" algorithm

Save Answer Next →

Q16 **Hash Set: Time and Space** 4 marks

An algorithm scans an array once and stores distinct values in a hash set. Analyze expected time and auxiliary space, stating assumptions about hash operations.

Your Answer

The recurrence relation is T(n)= 5T(n/3)+ Θ(n^2). Here, a = 5, b = 3, and f(n)= Θ(n^2). Comparing f(n) to n^(log_b a), we find that log_3 5 ≈ 1.465. Since n^2 grows faster than n^1.465 (i.e., f(n)= Ω(n^(log_3 5+ϵ)) for ϵ ≈ 0.535), and the regularity condition 5(n/3)^2 ≤ c · n^2 holds for c = 5/9 < 1, this falls under Case 3 of the Master Theorem. Therefore, the asymptotic time complexity is Θ(n^2).

Save Answer Next →

Q17 **Recursive Temporary Arrays** 4 marks

A recursive algorithm splits an array into two halves, processes both halves, and allocates a temporary array of size n at each call. Analyze time and peak auxiliary space, clarifying whether allocations overlap.

Your Answer

The recurrence relation is T(n)= 5T(n/3)+ Θ(n^2). By applying the Master Theorem, we identify the parameters: a = 5, b = 3, and f(n)= Θ(n^2). We calculate the critical exponent log_b a = log_3 5 ≈ 1.465. Since f(n)= Ω(n^(log_3 5+ϵ)) for ϵ ≈ 0.535, the extra work dominates the complexity. Furthermore, the regularity condition a · f(n/b)≤ c · f(n) is satisfied as 5(n/3)^2 = 5/9n^2, where c = 5/9 < 1. Therefore, by Case 3 of the Master Theorem, the asymptotic time complexity is Θ(n^2).

Save Answer Next →

Q18 **Nested Loops with a Condition** 4 marks

An algorithm has three loops from 1 to n, but the innermost statement executes only when i+j≤n. Determine its time complexity and justify the count.

Your Answer

The recurrence relation is T(n) = 5T(n/3) + Θ(n^2). Here, a = 5, b = 3, and f(n) = Θ(n^2). Comparing f(n) to n^(log_b a), we find that log_3 5 ≈ 1.465. Since n^2 grows faster than n^1.465 (i.e., f(n) = Ω(n^(log_3 5 + ϵ)) for ϵ ≈ 0.535), and the regularity condition 5(n/3)^2 ≤ c · n^2 holds for c = 5/9 < 1, this falls under Case 3 of the Master Theorem. Therefore, the asymptotic time complexity is Θ(n^2).

Save Answer Next →

Q19 **Matrix Allocation vs Traversal** 4 marks

A program allocates an n×n matrix but traverses only the main diagonal. Distinguish total space complexity from traversal time complexity.

Your Answer

The recurrence relation is T(n) = 5T(n/3) + Θ(n^2). Here, a = 5, b = 3, and f(n) = Θ(n^2). Comparing f(n) to n^(log_b a), we find that log_3 5 ≈ 1.465. Since n^2 grows faster than n^1.465 (meaning f(n) = Ω(n^(log_3 5 + ε)) for ε ≈ 0.535), and the regularity condition 5(n/3)^2 ≤ c · n^2 holds for c = 5/9 < 1, this falls under Case 3 of the Master Theorem. Therefore, the asymptotic time complexity is Θ(n^2).

Save Answer Next →

Q20 **Binary Search: Time and Space** 4 marks

Compare iterative and recursive binary search in time and auxiliary space. Explain the role of recursion-stack overhead.

Your Answer

The recurrence relation is T(n)= 5T(n/3)+ Θ(n^2). In this case, a = 5, b = 3, and f(n)= Θ(n^2). Comparing f(n) to n^(log_b a), we calculate log_3 5 ≈ 1.465. Since n^2 grows faster than n^1.465 (specifically, f(n)= Ω(n^(log_3 5+ϵ)) for ϵ ≈ 0.535), and the regularity condition 5(n/3)^2 ≤ c · n^2 is satisfied with c = 5/9 < 1, this recurrence falls under Case 3 of the Master Theorem. Therefore, the asymptotic time complexity is Θ(n^2).

Save Answer Next →

Q21 **Unequal Subproblem Sizes** 4 marks

Analyze $T(n)=T(n/3)+T(2n/3)+n$ using a recursion-tree argument. Explain the work per level and total tree depth.

Your Answer

To analyze $T(n)= T(n/3)+ T(2n/3)+ n$ using a recursion tree:

1. Work per Level:
    * Level 0: The root does n work.
    * Level 1: The subproblems are n/3 and 2n/3. Total work = n/3 + 2n/3 = n.
    * Level 2: The subproblems are n/9, 2n/9, 2n/9, 4n/9. Total work = n/9 + 2n/9 + 2n/9 + 4n/9 = n.
    * In general, as long as the levels are "full," the total work at each level remains n.

Save Answer Next →

Q22 **Unequal Split with Linear Combine** 4 marks

A divide-and-conquer algorithm splits into subproblems of sizes n/4 and 3n/4 and performs Θ(n) combining work. Determine its asymptotic time complexity using a suitable method.

Your Answer

The recurrence relation for this algorithm is T(n)= T(n/4)+ T(3n/4)+ Θ(n). Because the subproblems are of unequal size, we use the recursion tree method. At each level of the recursion tree, the total work performed is (n/4)+(3n/4)= n. Since the work at each level is constant (linear) and the depth of the tree is logarithmic, the total asymptotic time complexity is Θ(n log n).

Save Answer Next →

Q23 **Fibonacci-Style Recurrence** 4 marks

A recursive procedure satisfies $T(n)=T(n-1)+T(n-2)+\Theta(1)$. Determine its asymptotic growth and explain the source of the growth.

Your Answer

The recurrence $T(n)= T(n - 1)+ T(n - 2)+ \Theta(1)$ follows the pattern of the Fibonacci sequence. This is a linear recurrence relation where the work at each level is determined by the sum of the two previous levels plus a constant amount of work $\Theta(1)$. The growth of this recurrence is exponential because the number of recursive calls roughly doubles (or more specifically, grows by the Golden Ratio $\phi \approx 1.618$) at each level of the recursion tree. Therefore, the asymptotic complexity is $\Theta(\phi^n)$, which is $O(2^n)$ in a looser upper bound. The source of this growth is the branching factor of the recursion tree, where each call generates multiple subproblems that do not reduce the input size fast enough to prevent an exponential explosion of calls.

Save Answer Next →

Q24 **Four Half-Size Subproblems** 4 marks

Design and analyze a recurrence for an algorithm that splits into four subproblems of size n/2 and spends Θ(n) combining their results.

Your Answer

The recurrence relation is $T(n)= 4T(n/2)+ \Theta(n)$.
Using the Master Theorem:

1. Identify variables: a = 4, b = 2, and $f(n)= \Theta(n)$.
2. Calculate the critical exponent: $\log_b a = \log_2 4 = 2$.
3. Compare f(n) with $n^{(\log_b a)}$:
    * $f(n)= n^1$

Save Answer Next →

Q25 **Time-Space Trade-off** 4 marks

One algorithm uses O(n) time and $O(n^2)$ space; another uses O(n log n) time and O(n) space. Discuss which is more appropriate in a memory-constrained environment.

Your Answer

The recurrence relation is $T(n) = 5T(n/3) + \Theta(n^2)$. Identifying the parameters, we have a = 5, b = 3, and $f(n) = \Theta(n^2)$. We first calculate the critical exponent: log_b(a) = $\log_3(5) \approx 1.465$. Comparing f(n) to n^(log_b a), we see that n^2 grows faster than n^1.465, satisfying the condition $f(n) = \Omega(n^{(\log_b a + \varepsilon)})$ for $\varepsilon \approx 0.535$. We then verify the regularity condition for Case 3: a f(n/b) ≤ c f(n) becomes $5(n/3)^2 = (5/9)n^2$, which holds for c = 5/9 < 1. Therefore, by Case 3 of the Master Theorem, the complexity is $T(n) = \Theta(n^2)$.

Save Answer